

gflow Graph Low-Pass GCV Operator Used by `fit.rdgraph.regression()` and `refit.rdgraph.regression()`

SIMODS manuscript provenance note

Built 2026-05-15 01:56:59 EDT

1 Purpose and Manuscript Role

This note describes the graph low-pass smoothing operator implemented in `gflow` by `fit.rdgraph.regression()` and reused by `refit.rdgraph.regression()`. In the SIMODS non-oracle graph-selection benchmark, this operator is intended to serve as one candidate selector: for each candidate graph, fit a graph-spectral smoother, record its generalized cross-validation (GCV) score, and compare candidate graphs by aggregating those scores over selected feature signals.

Operationally, the benchmark should construct the graph/operator with `fit.rdgraph.regression()`. When the same fitted graph geometry and spectral filter are to be applied to many response or feature columns, the benchmark should call `refit.rdgraph.regression()` on the fitted object. The `refit` path reuses the cached graph geometry, eigenvectors, eigenvalues, and filter family. Depending on `per.column.gcv`, it either reuses the original smoothing parameter and filtered eigenvalues stored in the fitted model or selects a feature-specific smoothing parameter in the cached eigenbasis. Either way, it avoids rebuilding the Riemannian graph geometry for every feature.

Current implementation convention confirmed from source. For the current `fit.rdgraph.regression()` low-pass operator path with a precomputed graph supplied through `adj.list` and `weight.list`, `weight.list` is interpreted as a positive edge-length list used for local neighborhood ordering and truncation, not as an affinity or conductance list. The implementation does *not* directly use

$$a_{ij} = \frac{1}{\ell_{ij} + \epsilon}$$

as the smoothing edge weight. Instead, the supplied edge lengths ℓ_{ij} are used to order local graph neighborhoods. The smoothing operator is then built from neighborhood-intersection densities computed by the Riemannian-complex initialization; the conductance used in the mass-symmetrized spectral Laplacian is

$$c_e = \frac{1}{\max(\rho_1(e), 10^{-10})},$$

where $\rho_1(e)$ is the edge mass/density computed from neighborhood intersections. This is the key convention for the manuscript and benchmark.

2 Graph Input and Weight Semantics

Let $G = (V, E)$ be an undirected connected graph on $n = |V|$ vertices. In the R API, a precomputed graph is supplied through:

- `adj.list`: a length- n list whose i th entry gives the 1-based neighbors of vertex i ;
- `weight.list`: a length- n list parallel to `adj.list` whose entries are positive edge lengths ℓ_{ij} .

The R wrapper validates that `adj.list` and `weight.list` are supplied together, have matching lengths, contain no self-loops or duplicate neighbors, are undirected, and have reciprocal edge weights agreeing within tolerance. It then converts the adjacency list to 0-based indices for C++ while passing the numeric weights through as doubles.

The supported R-level precomputed-graph API currently requires `use.counting.measure = TRUE`. The wrapper explicitly stops if a precomputed `adj.list/weight.list` is supplied with `use.counting.measure = FALSE`. Therefore, in the benchmark usage envisioned here, ℓ_{ij} does not define a distance-based vertex reference measure; it defines edge lengths used to sort local neighborhoods.

2.1 From Edge Lengths to Local Neighborhoods

In the precomputed-graph C++ path, each row i is converted into a sorted list of pairs (ℓ_{ij}, j) , with a self-entry $(0, i)$ inserted. The implementation keeps the first $\min(k_{\text{cpp}}, \deg(i) + 1)$ entries as the local neighborhood $\hat{N}_k(i)$. Because the R wrapper passes `k + 1L` into C++ to account for the self neighbor convention, a user-level `k=K` corresponds to self plus up to K shortest graph-neighbors when the supplied degree is at least K .

Thus edge lengths enter the operator through neighborhood ordering and truncation:

$$\hat{N}_K(i) = \{i\} \cup \text{the up to } K \text{ adjacent vertices with smallest } \ell_{ij}.$$

The graph's full edge registry still contains all supplied undirected edges; the local neighborhoods are used to compute density and intersection masses.

2.2 Recent GCV Graph-Selection Regime

In the current supported precomputed-graph path, and in the recommended SIMODS graph-selection benchmark regime, calls to `fit.rdgraph.regression()` use the default counting-measure regime:

- `use.counting.measure = TRUE;`
- `response.penalty.exp = 0;`
- `density.alpha = 1.5;`
- `density.epsilon = 1e-10.`

The first and fourth values are especially easy to misread. In this precomputed graph path, `density.epsilon` is not an ℓ_{ij} -to-conductance regularizer. It belongs to the optional distance-reference branch $(\epsilon + d_k)^{-\alpha}$, which is not used when `use.counting.measure = TRUE`. The setting `response.penalty.exp = 0` disables response-coherence modulation, so the operator is the initialization-derived graph smoother rather than an iteratively response-adapted smoother.

Some historical or pre-run scripts experimented with `use.counting.measure = FALSE`. Current `gflow` rejects that setting for precomputed `adj.list/weight.list` graphs, so those historical calls should not be used as canonical method descriptions for the current precomputed-graph operator.

In this default counting-measure regime, for a graph edge $e = (i, j)$:

1. The local neighborhoods $\hat{N}_k(i)$ and $\hat{N}_k(j)$ are built from graph neighbors sorted by the supplied positive edge lengths, with the vertex itself included as a zero-distance self entry.
2. Each vertex has reference mass one, up to the global normalization convention for the reference measure.
3. The raw edge mass is essentially the local-neighborhood overlap,

$$\rho_{1,\text{raw}}(i, j) = \left| \hat{N}_k(i) \cap \hat{N}_k(j) \right|.$$

More generally, the code computes

$$\rho_{1,\text{raw}}(i, j) = \sum_{v \in \hat{N}_k(i) \cap \hat{N}_k(j)} \mu(v),$$

and $\mu(v) = 1$ in the counting-measure case.

4. The implementation then globally normalizes edge masses to the number of undirected edges, applies density floors/clamps for numerical stability, and stores the resulting symmetric edge density as $\rho_1(e)$.
5. The conductance used by the mass-symmetrized spectral operator is the inverse of this normalized/floored edge mass:

$$c_e^\rho = \frac{1}{\max(\rho_1(e), 10^{-10})}.$$

This description should be read as specific to the default counting-measure/precomputed-graph regime. Other parameter settings can change the reference measure or density construction, and response modulation can change edge masses during fitting.

3 Operator Construction

3.1 Reference Measure and Densities

For the supported precomputed-graph path, the reference measure is the counting measure:

$$\mu(i) = 1$$

up to the implementation's normalization convention. The C++ implementation also contains a distance-based reference-measure branch,

$$\mu(i) \propto (\epsilon + d_k(i))^{-\alpha},$$

with optional clamping and dynamic-range control, but this branch is not available through the R precomputed-graph API because the wrapper requires `use.counting.measure = TRUE` when `adj.list/weight.list` are provided.

The initial vertex and edge densities are computed from local neighborhoods and local-neighborhood intersections:

$$\rho_0(i) = \sum_{v \in \hat{N}_K(i)} \mu(v), \quad \rho_{1,\text{raw}}([i, j]) = \sum_{v \in \hat{N}_K(i) \cap \hat{N}_K(j)} \mu(v).$$

The code normalizes vertex densities to sum to the number of vertices and edge densities to sum to the number of undirected edges, applies floors/clamps for numerical stability, and stores the resulting ρ_1 edge densities symmetrically for both endpoints.

3.2 Mass Matrices

The vertex mass matrix is diagonal:

$$M_0 = \text{diag}(m_1, \dots, m_n), \quad m_i = \max(\rho_0(i), 10^{-8}).$$

The edge mass matrix M_1 is populated from edge densities. In the diagonal-only case, which is the default whenever triangle construction is not active, the diagonal entry for edge $e = [i, j]$ is

$$M_1(e, e) = \max(\rho_1(e), 10^{-15}).$$

When triangle cofaces are built, the implementation may also add off-diagonal entries to M_1 using triple neighborhood intersections:

$$M_1(e_{ij}, e_{is}) = \sum_{v \in \hat{N}_K(i) \cap \hat{N}_K(j) \cap \hat{N}_K(s)} \rho_0(v),$$

for edge pairs incident in a triangle. If triangle cofaces are absent, too large, or inconsistent, the code falls back to a diagonal edge mass approximation.

3.3 Boundary Operator and Hodge Laplacian

Let $B_1 \in \mathbb{R}^{n \times |E|}$ be the oriented vertex-edge incidence matrix. For an edge $e = (i, j)$ with $i < j$,

$$B_1(i, e) = -1, \quad B_1(j, e) = +1.$$

The general Hodge 0-Laplacian assembled by `laplacian_bundle_t` has the up-term

$$L_0 = B_1 M_1^{-1} B_1^\top M_0,$$

with the inverse metric applied by helper routines. This object is retained in the implementation as `L[0]`.

3.4 Actual Spectral Operator Used for Smoothing

The spectral decomposition used for smoothing preferentially uses a mass-symmetrized vertex Laplacian $\tilde{L}_0$, named `L0_mass_sym` in the implementation:

$$\tilde{L}_0 = M_0^{-1/2} L_{\text{div}} M_0^{-1/2}, \quad L_{\text{div}} = B_1 C B_1^\top.$$

Here $C = \text{diag}(c_e)$ is the diagonal conductance matrix and

$$c_e = \frac{1}{\max(\rho_1(e), 10^{-10})}.$$

The code comments describe this as the DEC convention $c_e = 1/m_1(e)$, so a larger edge mass implies weaker penalization of variation across that edge. If `L0_mass_sym` is unavailable, the eigensolver falls back to `L[0]`, but the normal path is the mass-symmetrized operator.

This distinction matters for manuscript wording. A simple graph-Laplacian intuition is acceptable if stated as an intuition: shorter supplied edge lengths affect which vertices enter the local neighborhood intersections, which in turn affect edge masses and conductances. The implementation is not a simple direct weighted Laplacian with $w_{ij} = 1/(\ell_{ij} + \epsilon)$.

4 Methodological Caveat and Recommended Comparison

The current `fit.rdgraph.regression()` operator should therefore be described as an overlap-density/Riemannian-complex smoother. It is not the most standard metric-length graph smoother. For the SIMODS geodesic-data-geometry manuscript, this distinction matters because the non-oracle graph-selection objective is meant to assess graph metric geometry. A more standard graph-signal alternative would construct conductance directly from the supplied edge length, for example

$$c_e^{\text{len}} = \frac{1}{\ell_e + \epsilon}.$$

That length-based conductance rule should be benchmarked as an alternative smoother, not presented as the current behavior of `fit.rdgraph.regression()`.

For future non-oracle graph-parameter selection experiments, we recommend a controlled comparison between the current overlap-density conductance

$$c_e^\rho = \frac{1}{\max(\rho_1(e), 10^{-10})}$$

and a direct length-based conductance

$$c_e^{\text{len}} = \frac{1}{\ell_e + \epsilon}.$$

The key benchmark question is whether these two smoothers select graph parameters that agree with oracle geodesic-isometry performance on synthetic quad benchmarks. This comparison would separate two scientific questions that are currently entangled: whether a candidate graph has good local overlap geometry under the Riemannian-complex construction, and whether its metric edge lengths support good graph-signal smoothing.

5 Spectral Low-Pass Filter

Let (λ_j, v_j) , $j = 1, \dots, m$, be the cached eigenpairs of the operator used for smoothing, normally $\tilde{L}_0$. Let

$$V = [v_1, \dots, v_m], \quad \Lambda = \text{diag}(\lambda_1, \dots, \lambda_m).$$

For a fully observed response $y \in \mathbb{R}^n$, the smoother evaluates

$$y_{\text{spec}} = V^\top y, \quad \hat{y}_\eta = V h_\eta(\Lambda) V^\top y,$$

where $h_\eta(\Lambda)$ is diagonal with entries $h_\eta(\lambda_j)$.

The supported filter families are:

$$\begin{aligned}
\text{heat_kernel} : \quad & h_\eta(\lambda) = \exp(-\eta\lambda), \\
\text{tikhonov} : \quad & h_\eta(\lambda) = \frac{1}{1 + \eta\lambda}, \\
\text{cubic_spline} : \quad & h_\eta(\lambda) = \frac{1}{1 + \eta\lambda^2}, \\
\text{gaussian} : \quad & h_\eta(\lambda) = \exp(-\eta\lambda^2), \\
\text{exponential} : \quad & h_\eta(\lambda) = \exp(-\eta\sqrt{\max(\lambda, 0)}), \\
\text{butterworth} : \quad & h_\eta(\lambda) = \frac{1}{1 + (\lambda/\eta)^4}.
\end{aligned}$$

The R wrapper default is `filter.type = "heat_kernel"`.

6 GCV Criterion and Eta Grid

6.1 GCV in `fit.rdgraph.regression()`

For a fully labeled response, `fit.rdgraph.regression()` evaluates 40 candidate η values and selects the one minimizing

$$\text{GCV}(\eta) = \frac{\|y - \hat{y}_\eta\|_2^2}{(n - \text{tr}(S_\eta))^2}, \quad \text{tr}(S_\eta) = \sum_{j=1}^m h_\eta(\lambda_j).$$

The implementation floors the denominator at 10^{-10} in absolute value. For each candidate, it computes filter weights, applies them in the spectral domain, reconstructs $\hat{y}_\eta$, computes the GCV score, and retains the best candidate.

The C++ eta grid in `fit.rdgraph.regression()` is logarithmic with $\eta_{\min} = 10^{-10}$. Its $\eta_{\max}$ depends on the filter family and the largest retained eigenvalue $\lambda_{\max}$:

$$\eta_{\max} = \begin{cases} -\log(10^{-10})/\lambda_{\max}, & \text{heat kernel, Gaussian, exponential,} \\ 10^{10}, & \text{Tikhonov, cubic spline, Butterworth,} \\ 1, & \text{fallback.} \end{cases}$$

The grid is generated by exponential spacing between these bounds.

The fitter records the GCV result after the initial smoothing and after each geometric refinement iteration. The returned fitted values are selected from the iteration whose recorded GCV score is minimal. At the end, the fitter also stores the filter weights $h_{\eta^*}(\lambda_j)$ for the selected iteration.

6.2 Semi-Supervised GCV

When `y.vertices` is supplied to `fit.rdgraph.regression()`, the C++ smoother uses a masked spectral Tikhonov system rather than the direct $Vh_\eta(\Lambda)V^\top y$ formula. Let P be the diagonal mask for labeled vertices L , and let

$$A = V^\top PV, \quad b = V^\top Py.$$

For each η , it solves

$$(A + \eta\Lambda)a = b, \quad \hat{y}_\eta = Va,$$

and uses the labeled-set GCV criterion

$$\text{GCV}_L(\eta) = \frac{\|P(y - \hat{y}_\eta)\|_2^2}{(n_{\text{eff}} - \text{df}(\eta))^2}, \quad n_{\text{eff}} = |L|,$$

with

$$\text{df}(\eta) = \text{tr}\{(A + \eta\Lambda)^{-1}A\}.$$

7 Refit and Application to Multiple Features

`refit.rdgraph.regression()` expects a fitted object of class `knn.riem.fit`. It extracts:

$$V, \quad \lambda_1, \dots, \lambda_m, \quad h_{\eta^*}(\lambda_1), \dots, h_{\eta^*}(\lambda_m), \quad \eta^*.$$

With the default `per.column.gcv = FALSE` and no `y.vertices`, `refit` applies the cached linear smoother to each column of a new response matrix Y :

$$\hat{Y} = V \text{diag}(h_{\eta^*}(\lambda_1), \dots, h_{\eta^*}(\lambda_m)) V^\top Y.$$

The R implementation computes $V^\top Y$, multiplies each spectral row by the cached filter weight, and maps back by V . For large matrices, it can do this in column blocks.

If `per.column.gcv = TRUE`, `refit` does not reuse the original η^* . Instead, it reuses the cached eigenbasis and eigenvalues, builds an R-side eta grid, computes filter weights for each candidate, and selects an independent η_j^* for each response column by GCV. The R-side grid uses finite positive eigenvalues, with

$$\eta_{\text{lo}} = 1/\max(\lambda_j), \quad \eta_{\text{hi}} = 1/\min(\lambda_j),$$

and logarithmic spacing; it falls back to a linear grid from 10^{-3} to 1 if the eigenvalue range is unusable. Per-column GCV is disabled when `y.vertices` is supplied.

If `y.vertices` is supplied to `refit`, the default fixed- η path solves

$$(V_L^\top V_L + \eta^* \Lambda) a = V_L^\top y_L, \quad \hat{y} = V a,$$

for each column or column block. This is a semi-supervised fixed- η application, not a per-column GCV refit.

8 Implications for Non-Oracle Graph Selection

For graph selection, the recommended implementation-facing protocol is:

1. For each candidate graph G_r , pass `adj.list` and `weight.list` to `fit.rdgraph.regression()` with a proxy response or selected feature signal.
2. Ensure `weight.list` contains positive symmetric edge lengths, not affinities or pre-inverted conductances.
3. Extract the fitted model's GCV summary from `fit$gcv`; the fitted values correspond to the iteration with minimum GCV.
4. Apply `refit.rdgraph.regression()` to a matrix of selected feature signals. Use `per.column.gcv = TRUE` if the graph is to be scored by feature-specific smoothness parameters; otherwise use the cached original filter for all columns.

5. Aggregate the graph score over selected features, for example by the mean or median of per-column GCV scores when `per.column.gcv = TRUE`.

Manuscript-facing claims should describe this as a graph-spectral low-pass GCV criterion based on the `gflow` Riemannian graph smoother. The manuscript should not claim that edge lengths are directly transformed into affinities $1/(\ell + \epsilon)$. That formula is at best an intuition for inverse edge-mass conductance after the neighborhood-intersection construction, not the implemented edge-length conversion.

If the manuscript or benchmark wants a metric-length graph-smoothing selector, that should be implemented and documented as a distinct comparator using a conductance such as $c_e^{\text{len}} = 1/(\ell_e + \epsilon)$. The current `fit.rdgraph.regression()` selector should remain labeled as the overlap-density/Riemannian-complex GCV smoother.

9 Unresolved Ambiguities and Developer-Confirmation Items

- **Spectral operator versus full M_1 .** The generic Hodge operator $L[0]$ uses the metric machinery, while the spectral smoother normally decomposes `L0_mass_sym`, built from diagonal conductances $c_e = 1/\rho_1(e)$. If off-diagonal M_1 entries are enabled through triangle construction, those entries affect the generic $L[0]$ but do not appear to enter `L0_mass_sym` except through the diagonal conductance vector. This appears intentional from the source comments, but should be treated as an implementation detail unless the `gflow` developer confirms it.
- **Benchmark scoring convention.** The exact aggregation rule over feature columns, and whether to use fixed- η refit or per-column GCV, is a benchmark-design decision, not fixed by the package.
- **Response-coherence modulation.** The fitter can iteratively update edge masses based on the response. For graph-selection benchmarks seeking a graph-only criterion, the experiment design should decide whether to keep `response.penalty.exp = 0` so the graph geometry remains fixed after initialization.

10 Source Traceability

Mathematical or operational object	Code object		Source path and line region	Notes
R precomputed graph API	<code>adj.list</code> , <code>weight.list</code> ments	argu-	<code>/Users/pgajer/current_projects/gflow/R/riem_dcx_regression.R</code> , lines 30–36, 849–950	<code>weight.list</code> documented as edge-length list; validates positivity, symmetry, and reciprocal agreement.

Mathematical or operational object	Code object	Source path and line region	Notes
Default GCV selection regime	graph-function defaults	/Users/pgajer/current_projects/gflow/R/riem_dcx_regression.R, lines 654–681	Defaults include <code>response.penalty.exp = 0</code> , <code>use.counting.measure = TRUE</code> , <code>density.alpha = 1.5</code> , and <code>density.epsilon = 1e-10</code> .
Precomputed graph requires counting measure	R wrapper guard	/Users/pgajer/current_projects/gflow/R/riem_dcx_regression.R, lines 855–866	Supported R path stops if <code>use.counting.measure = FALSE</code> .
R-to-C++ call	<code>.Call(S_fit_rdgraph_regression, ...)</code>	/Users/pgajer/current_projects/gflow/R/riem_dcx_regression.R, lines 1384–1425	Passes 0-based adjacency and numeric weights to C++.
Registered C entry point	<code>S_fit_rdgraph_regression</code>	/Users/pgajer/current_projects/gflow/src/init.c, lines 329–332	Registers 42-argument C routine.
Precomputed graph extraction	<code>precomputed_adj_list</code> , <code>precomputed_weight_list</code>	/Users/pgajer/current_projects/gflow/src/riem_dcx_regression_r.cpp, lines 1429–1482	Copies weights but does not invert them.
Graph initialization dispatch	<code>fit_rdgraph_regression</code>	/Users/pgajer/current_projects/gflow/src/riem_dcx_regression.cpp, lines 5924–6013	Calls <code>initialize_from_graph()</code> when precomputed graph is supplied.
Edge length use in neighborhoods	<code>initialize_from_graph()</code>	/Users/pgajer/current_projects/gflow/src/riem_dcx_initialization.cpp, lines 209–262, 515–558	Sorts by supplied lengths and builds pseudo-kNN neighborhoods.
Reference measure	<code>initialize_reference_measure()</code>	/Users/pgajer/current_projects/gflow/src/riem_dcx_initialization.cpp, lines 1490–1665	Counting measure is uniform; optional distance branch uses $(\epsilon + d_k)^{-\alpha}$.
Vertex and edge densities	<code>compute_initial_densities</code>	/Users/pgajer/current_projects/gflow/src/riem_dcx_regression.cpp, lines 400–640	Defines ρ_0 and ρ_1 via neighborhood and inter-section masses.

Mathematical or operational object	Code object	Source path and line region	Notes
Mass matrices	<code>initialize_metric_from_dense()</code> <code>compute_edge_mass_matrices()</code>	<code>/Users/pgajer/current_projects/gflow/src/riem_dcx_regression.cpp</code> , lines 820–1068	Builds M_0 , M_1 , diagonal floors, and optional triple-intersection off-diagonals.
Boundary operator	<code>build_boundary_operator()</code>	<code>/Users/pgajer/current_projects/gflow/src/riem_dcx_regression.cpp</code> , lines 1070–1126	Builds oriented incidence matrix B_1 .
Conductance convention	<code>assemble_operators()</code>	<code>/Users/pgajer/current_projects/gflow/src/riem_dcx.cpp</code> , lines 289–307	Sets $c_e = 1/\max(\rho_1(e), 10^{-10})$.
Mass-symmetrized Laplacian	<code>build_L0_mass_sym_if_necessary()</code>	<code>/Users/pgajer/current_projects/gflow/src/laplacian_bundle.cpp</code> , lines 15–188	Builds $M_0^{-1/2} B_1 C B_1^\top M_0^{-1/2}$.
Generic Hodge assembly	<code>laplacian_bundle_t::assemble()</code>	<code>/Users/pgajer/current_projects/gflow/src/laplacian_bundle.cpp</code> , lines 190–245	Documents $B_{p+1} M_{p+1}^{-1} B_{p+1}^\top M_p$ and lower-dimensional term.
Spectral decomposition target	<code>compute_spectral_decomposition()</code>	<code>/Users/pgajer/current_projects/gflow/src/riem_dcx_regression.cpp</code> , lines 1679–1698	Uses <code>L0_mass_sym_if</code> if available, otherwise <code>L[0]</code> .
Filter formulas	<code>apply_filter_function()</code>	<code>/Users/pgajer/current_projects/gflow/src/riem_dcx_regression.cpp</code> , lines 3990–4028	Defines heat, Tikhonov, cubic spline, Gaussian, exponential, and Butterworth filters.
Fit-time GCV and eta grid	<code>compute_gcv_score()</code> , <code>generate_eta_grid()</code>	<code>/Users/pgajer/current_projects/gflow/src/riem_dcx_regression.cpp</code> , lines 4050–4115	Defines RSS over $(n - \text{tr } S)^2$ and C++ eta grid.
Fit-time smoother	<code>smooth_response_via_spline()</code>	<code>/Users/pgajer/current_projects/gflow/src/riem_dcx_regression.cpp</code> , lines 4123–4338	Computes eigenbasis projection, filter weights, GCV, and best η .
Semi-supervised smoother	<code>smooth_response_via_spline()</code>	<code>/Users/pgajer/current_projects/gflow/src/riem_dcx_regression.cpp</code> , lines 4360–4568	Uses masked Tikhonov system and labeled-set GCV.

Mathematical or operational object	Code object	Source path and line region	Notes
Cached spectral component	<code>create_spectral_component()</code>	<code>/Users/pgajer/current_projects/gflow/src/riem_dcx_regression_r.cpp</code> , lines 500–640	Exports eigenvalues, filtered eigenvalues, eigenvectors, optimal η , and filter type.
Fit result selected by GCV	result assembly	<code>/Users/pgajer/current_projects/gflow/src/riem_dcx_regression_r.cpp</code> , lines 2023–2060, 2266–2270	Selects fitted values from minimum-GCV iteration and returns spectral cache.
Fixed- η refit	<code>refit.rdgraph.regression_matrix()</code>	<code>/Users/pgajer/current_projects/gflow/R/refit_rdgraph_regression.R</code> , lines 111–115, 618–676	Applies cached V , eigenvalues, η^* , and filter weights to new columns.
Per-column GCV refit	<code>compute.filter.weights.select.eta.gcv.single()</code>	<code>/Users/pgajer/current_projects/gflow/R/refit_rdgraph_regression.R</code> , lines 270–306, 789–905	Reuses eigenbasis but selects per-column η with R-side GCV.
R-side η grid for refit	<code>generate.eta.grid()</code>	<code>/Users/pgajer/current_projects/gflow/R/compat_helpers.R</code> , lines 86–104	Uses $1/\lambda_{\max}$ to $1/\lambda_{\min}$ log grid for per-column refit.
High-level data smoother	<code>data.smoother()</code>	<code>/Users/pgajer/current_projects/gflow/R/data_smoother.R</code> , lines 1–25, 400–486	Selects graph scale by fit-time GCV, then calls refit on the full matrix.